Circuit and firmware to support Seiko-Epson G1216B1N000 dot graphics display

A serial interface and bias supply for the Seiko-Epson G1216N000 using an AT90S2313 because there just aren't enough applications examples for this display on the web.

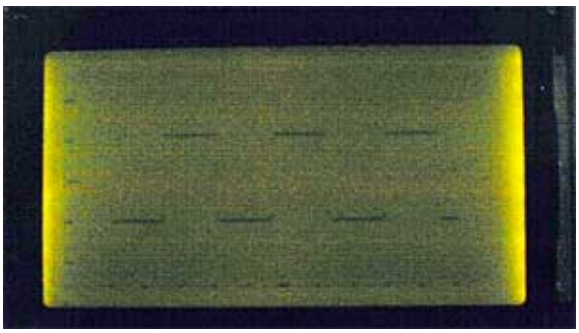
Under the hood of the display.

Download
Assembler source code

I was looking for an LCD display that I could use to display waveforms on the workbench. The selection criteria for the display module itself was straight forward:

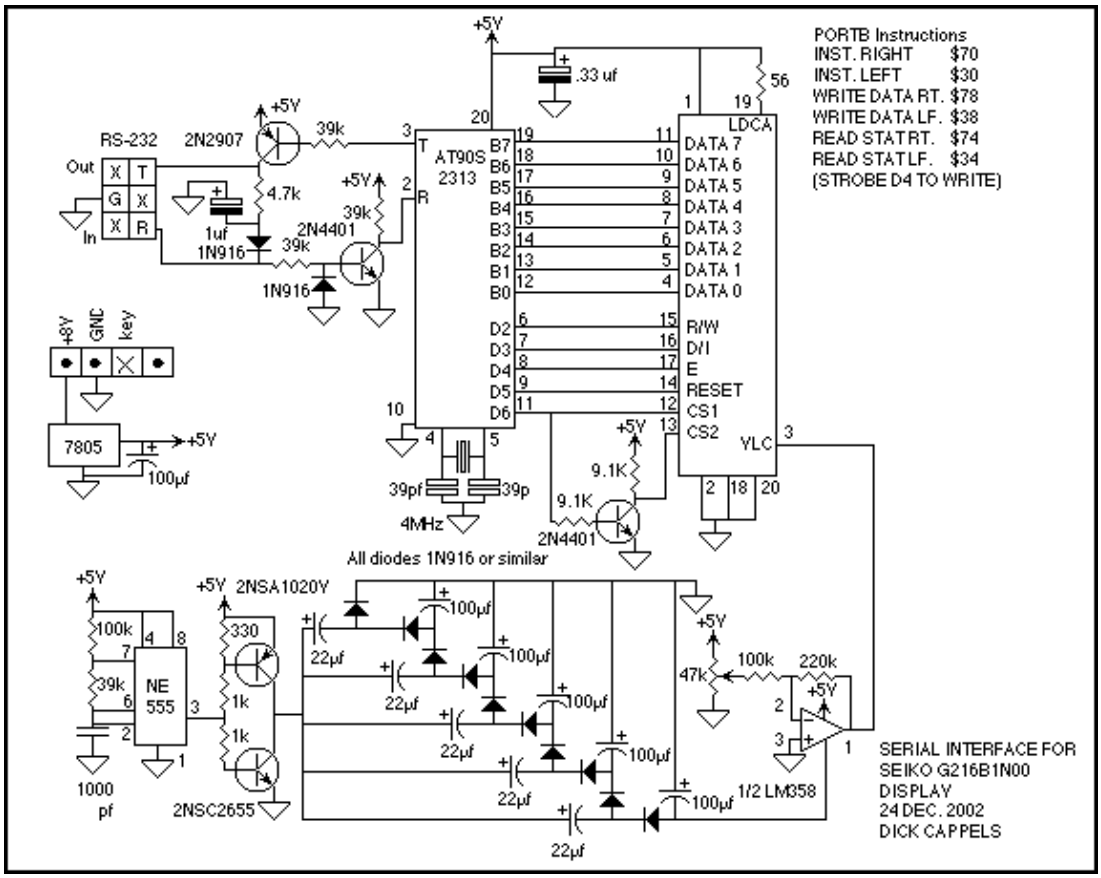
1. Dot graphic with sufficient resolution to display a simple waveform,
2. Available for fast delivery,
3. Have sufficient documentation available online to design with,
4. Be easy to interface to mechanically, electrically, and firmware wise, and
5. Have a low price.

The Seiko-Epson G1216B1N000 was selected, apparently meeting all criteria, but I was unable to get it to work until Jesper Hanse kindly sent me his driver for the driver for the display in his VAMPP4 TAMPP (Yet Another MP3 Player) and comparing his code with my code, I found some mistakes I had made.



Connected to the a waveform capture unit, displaying a 15 kHz square wave. I think the poor contrast has more to do with the digital camera's low light signal to noise ratio than the performance of the display.

The display is a pretty old transflective gray mode TN LCD design with ok, but not great contrast. Its contrast is better than the screen shot above. The display's saving grace is that appeared to meet all of the primary criteria. The code to drive it was complicated somewhat by the fact that the 8 bit interface appears as two 64x64 displays for the left and right hand sides, and since I was determined to control it with an AT90S2313, I was a pin short of being able to have separate select pins for the left and right displays, and so implemented a Right/Left control pin, which produced some artifacts in the display, and avoiding these made it necessary to carefully code around them.



More about the hardware:
The controller was implemented with an AT90S2313, but it could have been done with an AT90S1200 and a firmware UART simulation since code is pretty small and doesn't rely on the RAM or timers that an AT90S2313 provide. The hardware needs to drive the data and control lines of the Seiko-Epson module, supply an adjustable negative bias voltage and + 5 volts to the module, and provide inverting buffers for the RS-232 interface.

The inverting RS-232 takes its negative supply from the host. The negative supply could have come from the LCD bias circuit (see below) but the handy thing about this circuit was that its output swung negative when

connected to my RS-422 computer and only swung down to a little above ground when it was driven by the CMOS I/O port in the host on the waveform capture circuit that I connected it to after debugging.

The negative bias supply was generated by making 5V peak-to-peak pulses and using them to drive a 5 stage half-wave voltage multiplier and regulating the output with an opamp. The half-wave doubler is terribly inefficient using diodes for switch such a low voltage as each doubling of the 5 volts lost nearly half of it in the diodes, but I have a lot of diodes and miniature capacitors and wanted to pass the power signal through a single 5 volt regulator to minimize the exposure of the internal circuitry to accidental hazardous voltages on the cable. A single chip switched capacitor power supply would have reduced the parts count quite a bit.

The reference for the adjustable negative bias supply is the regulated +5 volt supply. The PNP and NPN saturating switches can be 2N2907 and 2N2222. The 330 Ohm resistor from the base of the PNP to +5V is necessary because the NPN emitter follower in the NE555 output stage cannot pull the output high enough to assure the PNP will shut off when the output is high.

More about the firmware:
After reset, the screen is erased, a line is drawn from the upper-left corner to the middle of the bottom edge, then another from there to the upper-right corner, which looks similar to a large letter "V" spanning the display. This is a visual indication that the display and serial controller are up and running. Its default is ASCII input mode, in which commands and the locations of dots to be written are received in ASCII- real nice for debugging with a terminal. Communications are at 19200 baud, 1 stop bit, no parity. The display can be switched to binary mode input, and in this mode commands and x,y coordinates are received as binary values, which is not easy to test with a terminal, but takes only half the transmission time to write a dot on the screen.

The ASCII conversion takes place within the GetASCIIorBinary routine, which returns a binary value regardless of whether the display is in the binary or ASCII input mode. If in the ASCII input mode, it also ignores commas, allowing it to graph comma delimited spreadsheet files.

The Epson documentation refers to the long axis as the Y axis and the coordinate 0,0 as being the upper-left corner of the screen (the latter is conventional for graphics systems). The driver converts this such that the coordinates 0,0 corresponds to the lower-left corner and 127,63 corresponds to the upper-right corner, which conventional for graphing.

The bits in a received byte (whether received as two characters in ASCII mode or a single byte in byte mode) are used according to the following rules:

Bit 7, if set, indicates that the byte is a command to be interpreted as one of the instructions listed below.

Bit 6 is the most significant bit of the Y address or 0 if it is an X address.

Bit 5 through 0 are the lower 5 bits of the X and Y addresses.

Four commands are supported by the serial controller:

$80 Erase display and reset

$81 Switch to ASCII input mode

$82 Switch to Binary input mode

$83 Restart the controller

Because of the way the Seiko-Epson module's interface works, dots are written in 7 row by 1 column segments (like 1 dot wide characters), and the current firmware doesn't read the display's internal ram to add a dot to it, rather it just writes a new segment of 1x7 dots, so this display is limited to simple graphs.

The assembly language source for the interface is **HERE.**

HOME

Please see the liability disclaimer on the HOME page.